

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«КРОС-ПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри АІС Бельська В. Ю.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 1

Тема: Основні керуючі оператори мови java. Масиви.

Мета: отримати навички у створенні консольних додатків з використанням основних керуючих операторів та масивів.

Порядок виконання роботи:

5. Гра 2048 — це логічна головоломка, де на полі 4×4 потрібно переміщувати клітинки ігрового поля з числами (ступенями двійки), об'єднуючи клітинки з однаковими числами для отримання клітинки з номіналом «2048».

Основна мета — об'єднувати клітинки, що розташовані поряд, сумуючи їх значення ($2+2=4$, $4+4=8$, $8+8=16\dots$), поки не буде досягнуто заповітної цифри. за наступними правилами:

Основні правила та механіка:

- Початок гри: на ігровій матриці розмірності 4×4 лише 2 довільні клітинки заповнені цифрою «2» - усі інші порожні (0 на екран не виводити)
 - Управління: використовуються стрілки на клавіатурі щоб рухати усі клітинки одночасно в один із чотирьох боків: вгору, вниз, вліво або вправо.
 - Об'єднання: коли дві клітинки з однаковим числом стикаються при русі, вони зливаються в одну, сума якої дорівнює їхньому загальному значенню.
 - Нові значення на полі: після кожного ходу одне нульове значення у масиві автоматично замінюється «2» (90% ймовірності) або «4» (10% ймовірності).
 - Програш: гра закінчується, якщо поле(усі комірки багатовимірною масиву) повністю заповнене і неможливо зробити хід (немає сусідніх однакових плиток для об'єднання).
 - Перемога: Ви досягаєте мети, коли отримуєте плитку з номіналом 2048, але можна продовжувати гру далі для встановлення рекорду.
 - Нарахування балів (score): на початку гри гравець має 0 балів.
- Впродовж гри бали збільшуються на величину числа, отриманого у результаті об'єднання клітинок, що мають одне значення.

Забезпечити гравцю можливість перервати гру, розпочати нову гру (багаторазове використання гри). На протязі використання програмного коду одним гравцем запам'ятовувати його найкращі результати з попередніх ігор при багаторазовому варіанті гри.

Варіант 15.

```
import java.util.Random;
import java.util.Scanner;

public class Game2048 {

    static int[][] field = new int[4][4];
    static int score = 0;
    static int bestScore = 0;
    static Random random = new Random();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean play = true;

        while (play) {

            initGame();

            while (true) {

                printField();

                if (!canMove()) {
                    System.out.println("Гра завершена! Нема можливих ходів.");
                    break;
                }

                System.out.println("Введіть хід (W-вгору, S-вниз, A-вліво, D-вправо, Q-вихід):");
                char move = sc.next().toUpperCase().charAt(0);

                boolean moved = false;

                if (move == 'A')
                    moved = moveLeft();
                if (move == 'D')
                    moved = moveRight();
                if (move == 'W')
                    moved = moveUp();
                if (move == 'S')
```

```

        moved = moveDown();
        if (move == 'Q')
            return;

        if (moved) {
            addRandomNumber();
        }
    }

    if (score > bestScore) {
        bestScore = score;
    }

    System.out.println("Ваш рахунок: " + score);
    System.out.println("Найкращий рахунок: " + bestScore);
    System.out.println("Почати нову гру? (Y/N)");

    if (!sc.next().equalsIgnoreCase("Y")) {
        play = false;
    }
}

static void initGame() {
    field = new int[4][4];
    score = 0;
    addRandomNumber();
    addRandomNumber();
}

static void printField() {

    System.out.println("\nРахунок: " + score + "    Найкращий: " + bestScore);

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

            if (field[i][j] == 0)
                System.out.print(".\t");
            else
                System.out.print(field[i][j] + "\t");
        }
        System.out.println();
    }
}

static void addRandomNumber() {

    int emptyCount = 0;

    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)

```

```

        if (field[i][j] == 0)
            emptyCount++;

    if (emptyCount == 0)
        return;

    int position = random.nextInt(emptyCount);
    int count = 0;

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

            if (field[i][j] == 0) {

                if (count == position) {
                    field[i][j] = (random.nextInt(10) < 9) ? 2 : 4;
                    return;
                }
                count++;
            }
        }
    }
}

static boolean moveLeft() {

    boolean moved = false;

    for (int i = 0; i < 4; i++) {

        for (int j = 1; j < 4; j++) {

            if (field[i][j] != 0) {

                int col = j;

                while (col > 0 && field[i][col - 1] == 0) {
                    field[i][col - 1] = field[i][col];
                    field[i][col] = 0;
                    col--;
                    moved = true;
                }

                if (col > 0 && field[i][col - 1] == field[i][col]) {
                    field[i][col - 1] *= 2;
                    score += field[i][col - 1];
                    field[i][col] = 0;
                    moved = true;
                }
            }
        }
    }
}

```

```

        return moved;
    }

    static boolean moveRight() {

        rotateHorizontal();
        boolean moved = moveLeft();
        rotateHorizontal();

        return moved;
    }

    static boolean moveUp() {

        transpose();
        boolean moved = moveLeft();
        transpose();

        return moved;
    }

    static boolean moveDown() {

        transpose();
        rotateHorizontal();
        boolean moved = moveLeft();
        rotateHorizontal();
        transpose();

        return moved;
    }

    static void rotateHorizontal() {

        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 2; j++) {

                int temp = field[i][j];
                field[i][j] = field[i][3 - j];
                field[i][3 - j] = temp;
            }
        }
    }

    static void transpose() {

        for (int i = 0; i < 4; i++) {
            for (int j = i + 1; j < 4; j++) {

                int temp = field[i][j];
                field[i][j] = field[j][i];

```

```

        field[j][i] = temp;
    }
}

static boolean canMove() {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {

            if (field[i][j] == 0)
                return true;

            if (j < 3 && field[i][j] == field[i][j + 1])
                return true;

            if (i < 3 && field[i][j] == field[i + 1][j])
                return true;
        }
    }

    return false;
}
}

```

```

Рахунок: 52   Найкращий: 0
2      .      .      2
.      .      .      .
.      .      .      16
.      .      4      2
Введіть хід (W-вгору, S-вниз, A-вліво, D-вправо, Q-вихід):

```

Рисунок 1.1 – Результат роботи скрипту

Висновок: на цій лабораторній роботі ми отримали навички у створення консольних додатків з використанням основних керуючих операторів та масивів, реалізували гру 2048 на java.